
Playbook — Design, UI e Mídia Soberana

Guia de bancada · 8 passos práticos

Heverton Eduardo Peres

Objetivo do Material

Executar, do início ao fim, os passos práticos de Design, UI e Mídia Soberana sem reler a teoria.

Como usar este playbook

Você é o **Praticante**. Cada passo é um card independente com sete partes: objetivo, pré-requisito, entregas, execução, gate de verificação, critério de conclusão e armadilhas.

Este documento **não repete a teoria** do livro. Quando precisar do porquê, siga a referência cruzada do card para o capítulo correspondente.

Mapa dos Estágios

#	Estágio	Passos
1	Estágio 1	1, 2, 3, 4
2	Estágio 2	5, 6, 7, 8

Passos Práticos

Passo 1 — Introdução à Soberania Tecnológica no Design e UI

Estágio: Estágio 1 • **Origem:** Cap. 1 — Introdução à Soberania Tecnológica no Design e UI

① Objetivo do passo

Imagine acordar em uma segunda-feira decisiva de entrega e descobrir que a plataforma em nuvem onde você criou toda a identidade visual da sua empresa alterou os Termos de Serviço da noite para o dia, reajustou o valor da assinatura em quatro vezes ou simplesmente bloqueou o...

② Pré-requisito

Nenhum — este é o ponto de partida

③ Entregas

- `docker-compose.yml`
- `.env`
- `@fontsource/inter`

④ Execução

Execução

```
version: '3.8'

services:
  # Plataforma Aberta de Design e UI Prototyping
  penpot-frontend:
    image: penpotapp/frontend:latest
    container_name: penpot_frontend
    ports:
      - "9001:80"
    environment:
      - PENPOT_FLAGS=enable-mimett-check
    networks:
      - soberano_net
    restart: unless-stopped

  # Utilitario Autonomo de Manipulacao de Documentos e PDF
  stirring-pdf:
    image: frooodle/s-pdf:latest
```

```

    container_name: stirring_pdf
    ports:
      - "8080:8080"
    environment:
      - DOCKER_ENABLE_SECURITY=false
      - SYSTEM_DEFAULTLOCALE=pt_BR
    networks:
      - soberano_net
    restart: unless-stopped

networks:
  soberano_net:
    driver: bridge

```

Execução

```

# Configuracoes de Infraestrutura de Design Soberano
PROJECT_NAME=design_soberano_local
DOMAIN_LOCAL=localhost
PENPOT_PORT=9001
STIRLING_PORT=8080
FONTSOURCE_CACHE_DIR=./cache/fonts
STORAGE_PATH=./dados_criativos

```

Execução

```

$ docker-compose up -d
[+] Running 3/3
  Container penpot_frontend   Started           0.8s
  Container stirring_pdf      Started           0.6s
  Network design_soberano_local_soberano_net   Created         0.1s

$ curl -I http://localhost:9001
HTTP/1.1 200 OK
Server: nginx
Date: Tue, 25 Aug 2026 14:00:00 GMT
Content-Type: text/html

$ curl -I http://localhost:8080/api/v1/info
HTTP/1.1 200 OK
Content-Type: application/json

```

Execução

```

# Instalacao local da fonte Inter via Fontsource
npm install @fontsource/inter

```

⑤ Verificação / Gate

```
docker-compose up -d
```

⑥ Feito quando...

- ☐ (a completar)

⑦ Armadilhas

- (a completar)

Passo 2 — A Geopolítica da Nuvem: Por que o software local importa?

Estágio: Estágio 1 • **Origem:** Cap. 2 — A Geopolítica da Nuvem: Por que o software local importa?

① Objetivo do passo

Imagine que todos os arquivos cruciais da sua empresa, desde a marca principal até os protótipos de produtos futuros, estejam armazenados em um cofre. Agora, imagine que esse cofre não fica na sua cidade, nem no seu país, mas em um servidor sujeito às leis e sanções comerciais...

② Pré-requisito

Passo 1 concluído

③ Entregas

- `iniciar-estudio-seguro.sh`

④ Execução

Execução

```
#!/bin/bash
# Script operacional para iniciar um ambiente de design seguro com rede isolada da internet
# Ferramenta orquestrada: Stirling-PDF para edição de documentos totalmente offline

echo "Verificando e criando rede Docker isolada (internal: true)..."
docker network create --internal rede_soberana_estudio || echo "A rede segura já está ativa."

echo "Iniciando o utilitário Stirling-PDF na rede hermética local..."
# O contêiner de documentação é conectado apenas à rede sem roteamento para a web aberta,
# impedindo que qualquer relatório comercial trafegue para IPs não autorizados.
docker run -d \
  --name pdf_soberano_offline \
  --network rede_soberana_estudio \
  -p 8080:8080 \
  -v $(pwd)/documentos_sigilosos:/configs \
  -e DOCKER_ENABLE_SECURITY=false \
  -e SYSTEM_DEFAULTLOCALE=pt_BR \
  frooodle/s-pdf:latest
```

```
echo "===== "  
echo "Ambiente de Editoração Soberana em execução com sucesso!"  
echo "Acesse o seu painel protegido de PDFs via navegador em: http://localhost:8080"  
echo "Nenhum arquivo processado aqui cruzará as fronteiras da sua rede local."  
echo "===== "
```

⑤ Verificação / Gate

```
docker network create --internal rede_soberana_estudio || echo "A rede segura já está  
ativa."
```

⑥ Feito quando...

- ☐ (a completar)

⑦ Armadilhas

- (a completar)

Passo 3 — Design Vetorial e UI: Penpot como alternativa ao Figma

Estágio: Estágio 1 • **Origem:** Cap. 3 — Design Vetorial e UI: Penpot como alternativa ao Figma

① Objetivo do passo

Como visto no Capítulo 2, onde exploramos a vulnerabilidade da geopolítica da nuvem e a urgência do software local, o design de interfaces digitais tornou-se uma das disciplinas mais estratégicas no desenvolvimento de produtos modernos. Ao dominar ferramentas de design vetorial...

② Pré-requisito

Passo 2 concluído

③ Entregas

- `docker-compose.yml`

④ Execução

4.1 Instalação via Docker Compose

```
mkdir penpot-server  
cd penpot-server
```

```
curl -o docker-compose.yml https://raw.githubusercontent.com/penpot/penpot/main/docker/images/docker-compose.yaml
```

4.2 Primeiro acesso e criação de projetos

```
docker exec -it penpot-postgres psql -U penpot -d penpot -c "UPDATE profile SET is_active = true WHERE email = 'seu_email@exemplo.com';"
```

⑤ Verificação / Gate

```
curl -o docker-compose.yml https://raw.githubusercontent.com/penpot/penpot/main/docker/images/docker-compose.yaml
```

⑥ Feito quando...

- ☐ Exportação estruturada do Figma:** Cada página do design system foi exportada como SVG, preservando camadas e nomes de objetos. Plugins de terceiros (como Figma to Code) foram usados para extrair metadados de design tokens (cores, espaçamentos, tipografia) em JSON [61]
- ☐ Reconstrução no Penpot:** Os SVGs foram importados como referência visual, mas os componentes foram redesenhados nativamente no Penpot para aproveitar Flexbox e Grid — o que não existia no Figma de forma nativa [62]. Tokens de cor e tipografia foram configurados manualmente no Assets Panel, seguindo a mesma nomenclatura do Figma (ex.: `color-brand-primary`, `font-size-lg`) para facilitar a transição dos designers [63]
- ☐ Validação colaborativa:** Cada designer recebeu acesso à instância auto-hospedada do Penpot e passou uma semana testando o novo ambiente em projetos não críticos. Problemas de compatibilidade (fontes faltantes, atalhos de teclado diferentes) foram documentados e resolvidos via customização do frontend do Penpot [64]

⑦ Armadilhas

- (a completar)

Passo 4 — Tipografia Sob Controle: Auto-hospedagem com Fontsource

Estágio: Estágio 1 • **Origem:** Cap. 4 — Tipografia Sob Controle: Auto-hospedagem com Fontsource

① Objetivo do passo

No Capítulo 3, vimos como o design vetorial autônomo com o Penpot nos liberta do lock-in corporativo e garante a nossa independência sobre os arquivos nativos de interface . Contudo, há um componente fundamental no design de interfaces contemporâneas que frequentemente é...

② Pré-requisito

Passo 3 concluído

③ Entregas

- `index.html`
- `package.json`
- `main.js`
- `_app.tsx`
- `index.ts`
- `global.css`

④ Execução

Execução

```
npm install @fontsource/roboto
```

Execução

```
// Importando o CSS que define a regra @font-face do peso 400 (Regular)
import '@fontsource/roboto/400.css';

// Importando o CSS que define a regra @font-face do peso 700 (Bold)
import '@fontsource/roboto/700.css';

import './global.css';

function Application() {
  return (
    <div className="soberano">
      <h1>Tipografia Rápida e Auto-Hospedada</h1>
      <p>Velocidade superior, zero impacto na privacidade.</p>
    </div>
  );
}

export default Application;
```

Execução

```
:root {
  /* Declaramos a fonte instalada como a raiz de todo documento */
  font-family: 'Roboto', system-ui, -apple-system, sans-serif;
  background-color: #0e0e11;
  color: #f0f0f5;
  line-height: 1.6;
}

h1 {
  font-weight: 700;
  letter-spacing: -0.02em;
}

p {
```

```
font-weight: 400;  
}
```

⑤ Verificação / Gate

```
npm install @fontsource/roboto
```

⑥ Feito quando...

- ☐ Você deleta sem hesitar as importações de CDN dos arquivos HTML raízes do projeto ativista
- ☐ Em sua máquina local, instala ambas as tipografias via NPM:

```
npm install @fontsource/fira-sans @fontsource/merriweather
```
- ☐ Injeta a importação estrita dos pesos cruciais no ponto de entrada global da aplicação
- ☐ Gera um novo deploy e envia para a infraestrutura de hospedagem criptografada da fundação [8]

⑦ Armadilhas

- (a completar)

Passo 5 — Upscaling Local: Real-ESRGAN e Upscayl para criativos

Estágio: Estágio 2 · **Origem:** Cap. 5 — Upscaling Local: Real-ESRGAN e Upscayl para criativos

① Objetivo do passo

Você já se deparou com aquela fotografia antiga, esmaecida pelo tempo, que gostaria de restaurar? Ou precisou ampliar um logotipo de baixa resolução para uma impressão profissional, apenas para ver os pixels se transformarem em manchas borradas? A ampliação de imagens sempre foi...

② Pré-requisito

Passo 4 concluído

③ Entregas

- `~/.config/Upscayl/models/`
- `_upscayled_4x.png`
- `input/upscayled/`

④ Execução

Configuração do ambiente Upscayl

```
# Linux (AppImage)
wget https://github.com/upscayl/upscayl/releases/download/v2.10.0/Upscayl-2.10.0.AppImage
chmod +x Upscayl-2.10.0.AppImage
./Upscayl-2.10.0.AppImage

# macOS (via Homebrew)
brew install --cask upscayl

# Windows (Instalador .exe)
# Baixe Upscayl-2.10.0.exe da página de releases e execute
```

Configuração avançada: Real-ESRGAN via linha de comando

```
# Instalação via pip (requer Python 3.8+)
pip install realesrgan

# Baixar modelos pré-treinados
wget https://github.com/xinntao/Real-ESRGAN/releases/download/v0.2.5.0/realesr-general-x4v3.pth \
-P ~/.cache/realesrgan/weights/

# Upscale único com modelo geral
realesrgan-ncnn-vulkan -i input.jpg -o output.png -n realesr-general-x4v3 -s 4

# Upscale em lote
for img in input/*.jpg; do
    realesrgan-ncnn-vulkan -i "$img" -o "output/${basename "$img"}.jpg_4x.png" \
    -n realesr-general-x4v3 -s 4
done
```

Integração com ferramentas de edição

```
git clone https://github.com/Aimarekin/Upscaler-for-GIMP.git
cp Upscaler-for-GIMP/upscaler.py ~/.config/GIMP/2.10/plugin-ins/
chmod +x ~/.config/GIMP/2.10/plugin-ins/upscaler.py
```

Otimização de performance

```
realesrgan-ncnn-vulkan -i input.jpg -o output.png -n realesr-general-x4v3 -s 4 -f fp16
```

⑤ Verificação / Gate

```
./Upscayl-2.10.0.AppImage
```

⑥ Feito quando...

- ☐ Diagnóstico**: verificar hardware disponível (workstation com RTX 3060, 12 GB VRAM)
- ☐ Configuração**: instalar Upscayl, baixar modelo RealESRGAN_x4plus
- ☐ Teste piloto**: processar 50 imagens de amostra, avaliar qualidade e ajustar modelo se necessário
- ☐ Produção em lote**: rodar processamento durante a madrugada (5-8 horas para 5.000 imagens)

- ☐ Controle de qualidade**: revisar aleatoriamente 5% dos resultados, reprocessar falhas pontuais com modelo alternativo
- ☐ Integração de pipeline**: adicionar etapa automática de upscaling antes da importação no After Effects
- ☐ Script de automação** (shell):

⑦ Armadilhas

- (a completar)

Passo 6 — Geração de Imagens: ComfyUI e o controle total da difusão

Estágio: Estágio 2 · **Origem:** Cap. 6 — Geração de Imagens: ComfyUI e o controle total da difusão

① Objetivo do passo

No Capítulo 5, você dominou o upscaling local com ferramentas como Real-ESRGAN e Upscayl, aprendendo a ampliar e restaurar ativos visuais diretamente no seu computador sem depender de APIs proprietárias ou servidores em nuvem. Essa capacidade de aprimorar imagens já existentes...

② Pré-requisito

Passo 5 concluído

③ Entregas

- `models/checkpoints/`

④ Execução

Instalação e Execução Local

```
# Clonar o repositório oficial da engine
git clone https://github.com/Comfy-Org/ComfyUI.git
cd ComfyUI

# Criar e ativar o ambiente virtual Python
python3 -m venv venv
source venv/bin/activate

# Instalar as dependências de aceleração de hardware (PyTorch e TorchVision)
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
pip install -r requirements.txt
```

```
# Iniciar o servidor local na porta padrao 8188
python3 main.py --listen 127.0.0.1 --port 8188
```

Estrutura do Grafo e Metadados no PNG

```
{
  "id": 3,
  "type": "KSampler",
  "inputs": {
    "seed": 428910482,
    "steps": 25,
    "cfg": 7.5,
    "sampler_name": "euler_ancestral",
    "scheduler": "karras",
    "denoise": 1.0,
    "model": ["1", 0],
    "positive": ["6", 0],
    "negative": ["7", 0],
    "latent_image": ["5", 0]
  },
  "outputs": {
    "LATENT": ["4", 0]
  }
}
```

⑤ Verificação / Gate

```
git clone https://github.com/Comfy-Org/ComfyUI.git
```

⑥ Feito quando...

- ☐ (a completar)

⑦ Armadilhas

- (a completar)

Passo 7 — Síntese de Voz: Kokoro e áudio neural local

Estágio: Estágio 2 · **Origem:** Cap. 7 — Síntese de Voz: Kokoro e áudio neural local

① Objetivo do passo

No Capítulo 6, você aprendeu a orquestrar pipelines visuais autônomos no ComfyUI , assumindo o controle total sobre a geração sintética de imagens sem depender de plataformas fechadas ou pagamentos por requisição. Essa autonomia gráfica é um pilar indispensável para o designer...

② Pré-requisito

Passo 6 concluído

③ Entregas

- `dist/áudio/`

④ Execução

Instalação e Execução via CLI

```
# Criar diretorio para o motor de audio neural local
mkdir -p ~/audio-soberano && cd ~/audio-soberano

# Baixar o executavel e o modelo neural leve da engine piper1-gpl
curl -L -O https://github.com/OHF-Voice/piper1-gpl/releases/download/v1.0.0/piper_linux_x86_64.tar.gz
tar -xzf piper_linux_x86_64.tar.gz

# Gerar o primeiro arquivo de audio de narração local
echo "Soberania tecnológica na síntese de voz neural." | ./piper --model pt_BR-faber-medium.onnx --output_file narracao.wav
```

Automação em Python para Pipelines de Mídia

```
import os
import subprocess
import time
from typing import Dict, Any

class SintetizadorVozSoberano:
    """
    Gerenciador local de síntese de voz neural baseado em piper1-gpl.
    """
    def __init__(self, caminho_binario: str, caminho_modelo: str):
        self.caminho_binario = caminho_binario
        self.caminho_modelo = caminho_modelo

        if not os.path.exists(self.caminho_binario):
            raise FileNotFoundError(f"Binario nao encontrado: {self.caminho_binario}")
        if not os.path.exists(self.caminho_modelo):
            raise FileNotFoundError(f"Modelo neural nao encontrado: {self.caminho_modelo}")

    def sintetizar(self, texto: str, arquivo_saida: str) -> Dict[str, Any]:
        """
        Sintetiza uma frase e retorna métricas de tempo de inferência.
        """
        inicio = time.perf_counter()

        comando = [
            self.caminho_binario,
            "--model", self.caminho_modelo,
            "--output_file", arquivo_saida
        ]
```

```

processo = subprocess.Popen(
    comando,
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    text=True,
    encoding="utf-8"
)

stdout, stderr = processo.communicate(input=texto)
fim = time.perf_counter()

duracao_execucao = fim - inicio
sucesso = processo.returncode == 0

return {
    "sucesso": sucesso,
    "tempo_segundos": round(duracao_execucao, 4),
    "arquivo": arquivo_saida,

```

⑤ Verificação / Gate

```
curl -L -O https://github.com/OHF-Voice/piper1-gpl/releases/download/v1.0.0/piper_linux_x86_64.tar.gz
```

⑥ Feito quando...

- ☐ (a completar)

⑦ Armadilhas

- (a completar)

Passo 8 — Documentação Soberana: Stirling-PDF e fluxos de trabalho autônomos

Estágio: Estágio 2 · **Origem:** Cap. 8 — Documentação Soberana: Stirling-PDF e fluxos de trabalho autônomos

① Objetivo do passo

No Capítulo 7, exploramos como o ecossistema Kokoro permitiu a emancipação completa da síntese de voz, substituindo infraestruturas dependentes de nuvem por uma solução neural leve e profundamente personalizável. Compreender essa transição em áudio pavimentou o caminho para uma...

② Pré-requisito

Passo 7 concluído

③ Entregas

- `docker-compose.yml`

④ Execução

Execução

```
import os
import requests

def executar_pipeline_ocr_soberano(diretorio_entrada: str, diretorio_saida: str,
url_base="http://localhost:8080"):
    """
    Executa Processamentos OCR batch consumindo a API REST do Stirling-PDF local.
    Garante a confidencialidade e elimina custos SaaS, permitindo a extração de
    texto em massa de PDFs escaneados ou rasterizados.
    """
    endpoint_ocr = f"{url_base}/api/v1/misc/ocr-pdf"

    if not os.path.exists(diretorio_saida):
        os.makedirs(diretorio_saida)
        print(f"Diretório de saída criado: {diretorio_saida}")

    arquivos_sucesso = 0
    arquivos_falha = 0

    print(f"Iniciando varredura no diretório: {diretorio_entrada}")
    for nome_arquivo in os.listdir(diretorio_entrada):
        if not nome_arquivo.lower().endswith(".pdf"):
            continue

        caminho_entrada = os.path.join(diretorio_entrada, nome_arquivo)
        caminho_saida = os.path.join(diretorio_saida, f"ocr_pesquisavel_{nome_arquivo}")

        print(f"Processando OCR em: {nome_arquivo}...")

        with open(caminho_entrada, 'rb') as arquivo_pdf:
            # O payload reflete as variáveis aceitas pela API do Stirling-PDF
            dados_formulario = {
                'languages': 'por',      # Definindo idioma primário como Português
                'sidecar': 'false',      # Não criar arquivo de texto .txt separado
                'deskew': 'true',        # Alinhar a inclinação de folhas escaneadas
                'clean': 'true',         # Limpar fundo escuro
                'cleanFinal': 'false',
```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Para assimilar o verdadeiro valor da documentação autônoma

- ☐ Imagine a situação rotineira: você acabou de receber de um cliente governamental uma dezena de manuais de especificações escaneados em baixa resolução
- ☐ A pressão do prazo está se esgotando
- ☐ Um profissional desavisado invariavelmente recorre ao padrão comodista: abre um serviço genérico como “I-Love-PDF” ou congêneres
- ☐ Ele arrasta os dez arquivos sensíveis contendo informações embargadas para uma janela do navegador
- ☐ O site exibe barras de progresso simuladas e
- ☐ Para realizar o OCR do lote inteiro

⑦ Armadilhas

- (a completar)

Checklist Mestre

Passo 1 — Introdução à Soberania Tecnológica no Design e UI

- ☐ (a completar)

Passo 2 — A Geopolítica da Nuvem: Por que o software local importa?

- ☐ (a completar)

Passo 3 — Design Vetorial e UI: Penpot como alternativa ao Figma

- ☐ Exportação estruturada do Figma:** Cada página do design system foi exportada como SVG, preservando camadas e nomes de objetos. Plugins de terceiros (como Figma to Code) foram usados para extrair metadados de design tokens (cores, espaçamentos, tipografia) em JSON [61]
- ☐ Reconstrução no Penpot:** Os SVGs foram importados como referência visual, mas os componentes foram redesenhados nativamente no Penpot para aproveitar Flexbox e Grid — o que não existia no Figma de forma nativa [62]. Tokens de cor e tipografia foram configurados manualmente no Assets Panel, seguindo a mesma nomenclatura do Figma (ex.: `color-brand-primary`, `font-size-lg`) para facilitar a transição dos designers [63]
- ☐ Validação colaborativa:** Cada designer recebeu acesso à instância auto-hospedada do Penpot e passou uma semana testando o novo ambiente em projetos não críticos. Problemas de compatibilidade (fontes faltantes, atalhos de teclado diferentes) foram documentados e resolvidos via customização do frontend do Penpot [64]

Passo 4 — Tipografia Sob Controle: Auto-hospedagem com Fontsource

- ☐ Você deleta sem hesitar as importações de CDN dos arquivos HTML raízes do projeto ativista
- ☐ Em sua máquina local, instala ambas as tipografias via NPM:

```
npm install @fontsource/fira-sans @fontsource/merriweather
```
- ☐ Injeta a importação estrita dos pesos cruciais no ponto de entrada global da aplicação
- ☐ Gera um novo deploy e envia para a infraestrutura de hospedagem criptografada da fundação [8]

Passo 5 — Upscaling Local: Real-ESRGAN e Upscayl para criativos

- ☐ Diagnóstico:** verificar hardware disponível (workstation com RTX 3060, 12 GB VRAM)
- ☐ Configuração:** instalar Upscayl, baixar modelo RealESRGAN_x4plus
- ☐ Teste piloto:** processar 50 imagens de amostra, avaliar qualidade e ajustar modelo se necessário
- ☐ Produção em lote:** rodar processamento durante a madrugada (5-8 horas para 5.000 imagens)
- ☐ Controle de qualidade:** revisar aleatoriamente 5% dos resultados, reprocessar falhas pontuais com modelo alternativo
- ☐ Integração de pipeline:** adicionar etapa automática de upscaling antes da importação no After Effects
- ☐ Script de automação** (shell):

Passo 6 — Geração de Imagens: ComfyUI e o controle total da difusão

- ☐ (a completar)

Passo 7 — Síntese de Voz: Kokoro e áudio neural local

- ☐ (a completar)

Passo 8 — Documentação Soberana: Stirling-PDF e fluxos de trabalho autônomos

- ☐ Para assimilar o verdadeiro valor da documentação autônoma
- ☐ Imagine a situação rotineira: você acabou de receber de um cliente governamental uma dezena de manuais de especificações escaneados em baixa resolução
- ☐ A pressão do prazo está se esgotando
- ☐ Um profissional desavisado invariavelmente recorre ao padrão comodista: abre um serviço genérico como “I-Love-PDF” ou congêneres
- ☐ Ele arrasta os dez arquivos sensíveis contendo informações embargadas para uma janela do navegador
- ☐ O site exibe barras de progresso simuladas e
- ☐ Para realizar o OCR do lote inteiro